Domain Model

Version: 1.0

Purpose

This document describes the business domain of Trading Platform Pro.
The Domain Model represents the core business concepts of the platform and remains independent from technical implementation details.

Domain Philosophy

The Domain layer contains the business knowledge of the application.
It defines:
• business entities
• business rules
• value objects
• aggregates
• domain services
• domain events
No infrastructure concerns are allowed inside the Domain.

Core Domains

Trading Platform Pro consists of several bounded domains.
Major domains include:
• Market
• Trading
• Portfolio
• Risk
• Orders
• Positions
• Strategies
• Reporting
• Configuration
• Decision Center
Each domain evolves independently while following common architectural principles.

Main Business Entities

Examples of entities:
• Account
• Portfolio
• Position
• Order
• Trade
• Instrument
• Strategy
• Watchlist
• Workspace
Entities have identity and lifecycle.

Value Objects

Examples:

• Price
• Quantity
• Currency
• Symbol
• Percentage
• Money
• Timestamp
Value Objects are immutable.

Aggregates

Typical aggregates include:
• Portfolio
• Strategy
• Order
• Position
Each aggregate protects its own business consistency.

Domain Services

Domain Services encapsulate business logic that does not naturally belong to a single entity.
Examples:
• Risk Evaluation
• Position Calculation
• Order Validation
• Strategy Evaluation

Domain Events

Business events communicate significant state changes.
Examples:
• OrderSubmitted
• OrderFilled
• PositionOpened
• PositionClosed
• StrategyActivated
• RiskLimitExceeded
Events are immutable.

Repository Interfaces

The Domain defines repository interfaces only.
Implementations belong to the Infrastructure layer.
Example:
```
PositionRepository
↓
SQLitePositionRepository
```

Business Rules

Business rules always remain inside the Domain.
Examples:
• position validation

• order validation
• risk calculation
• portfolio consistency
• strategy rules

Domain Independence

The Domain must never depend on:
• databases
• YAML
• JSON
• REST
• UI
• Logging
• Frameworks
• Broker APIs

Evolution

The Domain Model evolves continuously as new business capabilities are introduced.
New concepts should integrate into existing bounded contexts whenever possible.

Bounded Contexts

Each business domain represents an independent bounded context.
Communication between bounded contexts should occur through:
• domain events
• well-defined interfaces
• application services
Avoid direct coupling between unrelated domains.

Domain Invariants

Business invariants must always be enforced inside the Domain.
Examples:
• portfolio consistency
• valid position state transitions
• order lifecycle integrity
• risk limit enforcement
Application or Infrastructure layers must never bypass domain rules.

Domain Evolution

New business capabilities should:
• extend existing domains where appropriate
• avoid creating unnecessary domains
• preserve ubiquitous language
• maintain backward compatibility whenever practical

Domain Review Checklist

Before merging verify:
• business rules remain in Domain
• no infrastructure dependencies
• aggregates remain consistent

• value objects stay immutable
• repository interfaces only
• domain events remain business-oriented

Related Documents

• Architecture.md
• Infrastructure.md
• Product_Vision.md
• AGENTS.md